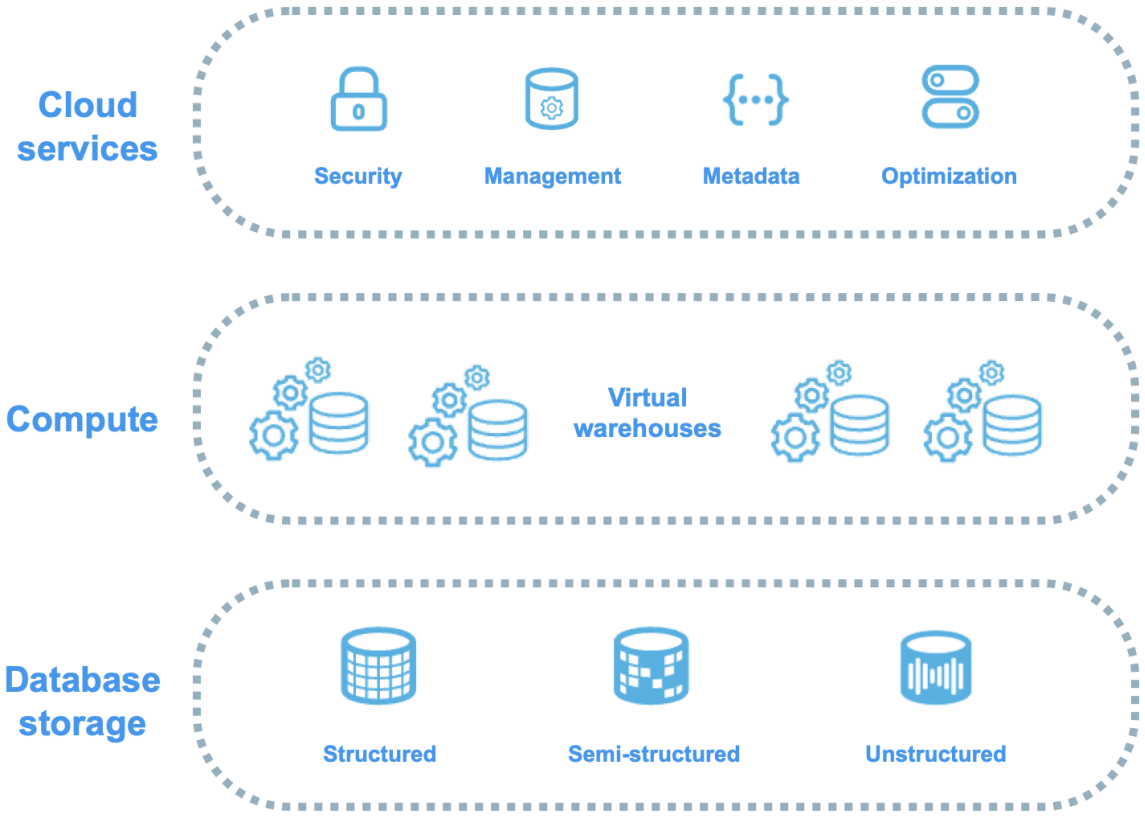
Snowflake - Professional Training
Facilitator: Ramkumar J Dhamodharan
https://www.linkedin.com/in/ramkumar-j-d-57423611/

Day Agenda

- Lecture on Snowflake Essentials
- Architecture
- Virtual Warehouse - Cost & Performance
- Database, Schema and Modeling
- Data Ingestion Techniques
    - Batch
    - Continuous
    - Streaming
    - Semi-Structured
- Demonstration & Practices

Snowflake Architecture

- In traditional warehouses, tie Storage and Compute together.
- Snowflake physically separates these into three different parts
    - Storage
    - Compute
    - Cloud Services
- Storage - Cloud Object Storage - Compressed - Immutable Files
- Compute - Virtual Warehouses - which are just clusters of computers you spin up and down.
- Cloud Services - The brain - Optimizer, Metadata, Security and Transaction Management



Analogies:

- Storage - Library
- Warehouses - Reading Rooms
- Cloud Services - Librarian

Micro-Partitions

- Small, Immutable Chunks of roughly 50 - 500 MB of uncompressed data each
- It remembers the min, max value of every column in every chunk
- WHERE trade_ts >= ‘2024-06-04’
- The phrase to remember here is that “PARTITIONS SCANNED vs. PARTITIONS TOTAL”

Warehouses

- The Cost & Performance Dial
- T-Shirt Size - XS, S, M, L, XL & Up
- Reason through
    - Credits Per Hour
    - Credits Per Query
- What size? And how long does it stay awake?

- Two Independent Knobs
    - Scale Up (More Power for a single heavy query)
    - Scale Out (Multi-Cluster - More copies of the same warehouse that spin up under concurrency)

- Analogy
    - Match the level to the symptom
        - One Slow Query -> Bigger Truck
        - A queue of waiting queries - More Trucks

- Two Settings
    - Auto-Suspend
    - Auto-Resume

The Lesson - Aggressive suspend is for SPIKY workloads with gaps - For a steady trickle, let it stay warm.

Database, Schemas and Modeling for Engineering

- Hierarchy
    - Account -> Database -> Schema -> Objects
        - (Tables, Views, Stages, Pipes, File Formats, Integrations …)
    - FQN (Fully Qualified Name - Database.Schema.Table)

- Layered Architecture
    - Medallion type
    - Raw | Staging | Analytics
    - Raw - Data lands here exactly as it arrives - source of truth
    - Staging - Cleaned | Typed | Deduplicated | Conformed (We trust this now - Layer)

- Analytics - Curated, Business-Ready Tables & Views that BI team and analysts actually query

Table Types

- Permanent - The Default - Has time-travel, Fail-Safe - Use for anything that matters
- Transient - No Fail-Safe, Shorter Time-Travel => Cheaper Storage
- Temporary - Vanishes when your session ends.

Can you reload the landing tables from S3 buckets? Yes, make them transient.

Snowflake's columnar storage just makes the wider dimensions join cheap.

OLTP systems - Row Storage
OLAP systems - Column Storage

Getting Data In

- The Bulk Loading Mental Model

Door	**Tool**	**When**	**Data Unit**
Batch Files / On-Demand	COPY INTO	Load this pile of files now	File(s)
Files, Continuously	Snowpipe	Load each file as it lands	File(s)
Rows, Continuously	Kafka | Snowpipe Streaming	Load events in near real-time	Row(s)

For bulk, the key concept is the stage - A pointer to where files live.

Storage Integration - Secure Handshake - Is a trust relationship between your snowflake account and an AWS IAM role.

Step by Step Process

- Create a storage integration in snowflake pointing at your bucket
- Run describe integration - which shall give you IAM User ARN and External ID
- In AWS, configure IAM role, trust policy and provide those two values
- Make sure that the role itself has S3 read permissions on the bucket
- In snowflake, create the stage that uses the integration

The three classic load failures

- When a load fails, it's almost always one of the three things
    - The file format does NOT match the file
    - A column count mismatch
    - A date/number that won't parse

- Load History (It remembers the metadata of what files have been loaded earlier)
- Load History Window

Semi-Structured Data - JSON without the pain

- VARIANT
- A single column that holds the entire JSON document (or array or any semi-structured value)
- SELECT raw:symbol::string, raw:price::float from market_ticks_json;

- LATERAL_FLATTEN - helps to explode an array of values into rows - turning one nested document into a proper table

- Load Now, define the shape later - This is SCHEMA-ON-READ

Snowpipe - Continuous File Ingestion

- Snowpipe is a COPY that fires automatically, the moment a file appears.
- Same load logic, but the trigger is the file's arrival, not your hand on the keyboard
- How does Snowpipe really work?
    - A file lands in the S3 bucket
    - S3 fires an event notification to an SQS queue (Snowflake gives you the Queue ARN to point your buckets' event notification at)
    - Snowpipe of Snowflake sees the message and runs the COPY command for that one file, using the pipe's predefined COPY command
    - Rows show up in your table seconds-to-minutes later - SERVERLESS
        - No Warehouse your manage | Pay to keep warm

- Snowpipe has a per-file cost component, so the shape of your files matters a lot.

Kafka & SnowPipe Streaming - True Real-Time

- How does it work?
    - Kafka topic carries the market-tick events
    - The Snowflake Sink Connector running in your Kafka cluster subscribes to the topic and writes straight into the Snowflake table
    - An option (ingestion method = SNOWFLAKE STEAMING vs. SNOWPIPE) - This configuration is the difference between rows-direct and the old-batch-into-files behavior
    - Authentication is KEY-PAIR (private and public key) - the connector signs in with the private key, and Snowflake verifies it against the public key you registered on the user.

Atlas - Fidelity Investments

- Fidelity's E&O (Equity and Options) Trading Desk is consolidating its market data estate into Snowflake.
- The codename for this platform is ATLAS
    - Must ingest market activity at three different velocities and serve it to analysts, risk and compliance from one place
        - Historical (Years of Equity Ticks for back-testing and TCA)
        - Batch (Nightly trade files dropped by venues (NYSE) into S3
        - Micro-Batch (Intraday tick files that land through the day and must be loaded within seconds)
        - Streaming (A live execution feed (orders/fills) flowing through Kafka, landing in Snowflake with low latency (Snowpipe Streaming)

- Along the way, the data is MESSY semi-structured JSON (nested option chains) that must be modeled into clean STAR schemas

Mermaid Diagram Contents

Browser -> https://mermaid.live

```
flowchart TB
    subgraph Sources["Market Data Sources"]
        NYSE["NYSE / NASDAQ<br/>nightly batch CSV"]
        CBOE["CBOE options<br/>nested JSON"]
        VENUE["intraday tick drops"]
        EXEC["live execution feed"]
    end

    subgraph AWS["AWS Landing Zone"]
        S3BATCH["S3 /batch (Lab 4)"]
        S3JSON["S3 /json (Lab 5)"]
        S3PIPE["S3 /autoingest (Lab 6)"]
        SQS["SQS queue"]
    end

    subgraph DOCKER["Local Docker (Lab 7)"]
        KAFKA["Kafka broker + Connect"]
    end

    subgraph SF["Snowflake DB: FIDELITY_ATLAS"]
        subgraph COMPUTE["Compute - Labs 1 & 2"]
            WH["Multi-cluster warehouses<br/>pruning-aware queries"]
        end
        subgraph RAW["RAW layer"]
            RTICKS["TICKS clustered (Lab 1)"]
            RTRD["RAW_TRADES (Lab 4)"]
            RJSON["RAW_OPTIONS VARIANT (Lab 5)"]
            RDROP["RAW_TICKS_STREAM (Lab 6)"]
            RSTREAM["RAW_EXECUTIONS (Lab 7)"]
        end
        subgraph MODEL["ANALYTICS layer (Lab 3)"]
            DIM["DIM_SECURITY / DIM_VENUE / DIM_DATE"]
            FACT["FACT_TRADES / FACT_QUOTES / FACT_EXECUTIONS"]
        end
    end

    subgraph CONS["Consumers"]
        AN["Trading analysts"]
        RK["Risk & compliance"]
```

```
end

NYSE --> S3BATCH --> RTRD
CBOE --> S3JSON --> RJSON
VENUE --> S3PIPE --> SQS --> RDROP
EXEC --> KAFKA --> RSTREAM

RTICKS --> MODEL
RTRD --> MODEL
RJSON --> MODEL
RDROP --> MODEL
RSTREAM --> MODEL
WH -.-> RAW
WH -.-> MODEL
MODEL --> AN
MODEL --> RK
```

The following are the capabilities we would like to experiment with.

- Pruning Experiments (Why are tick queries slow?)
- Scaling & Concurrency Test (Market open overwhelms the desk)
- The Star-Schema Modeling Challenge (Analysts can’t read raw feeds)
- Bulk Load + S3 Integration
- Semi-Structured JSON
- SnowPipe Auto-Ingest (SQS) - (Nightly is too slow)
- Kafka | Snowpipe Streaming (The execution feed must stream - real-time?)